P2A8. Algoritmos y su implementación

Diego Escobar Manrique
Estructuras de Datos y Algoritmos · 11 de noviembre de 2026

Introducción

Organizar información es un problema que existía mucho antes de las computadoras. Antes se usaban listas y fichas; ahora se usan bases de datos. Para comparar ambos casos elegí la búsqueda de libros en bibliotecas y la consulta de productos en una tienda en línea.

1. Problema antiguo: encontrar un libro en una biblioteca

¿En qué consiste? En saber si una biblioteca tiene un libro y dónde está, sin revisar todos los estantes.

¿Desde cuándo existe? El problema existe desde las primeras bibliotecas. Como caso concreto, la Biblioteca del Congreso utilizó catálogos impresos durante el siglo XIX y abrió su nuevo catálogo de fichas al público en 1900 [1].

¿Quiénes lo enfrentaban? Bibliotecarios, estudiantes, investigadores y lectores.

¿Qué dificultades había? Al crecer la colección, los catálogos impresos se desactualizaban. Además, una ficha fuera de lugar o un nombre mal escrito podía dificultar la búsqueda.

¿Cómo se resolvía originalmente? Se consultaban catálogos impresos y después fichas organizadas por autor o tema. La información del catálogo permitía identificar el libro y su ubicación [1, 2].

Relación con los algoritmos

- **Ordenamiento básico: inserción.** Para agregar una ficha, se busca su lugar y se coloca entre las fichas que ya están ordenadas. Es fácil de aplicar y funciona bien cuando llegan pocos libros. Su desventaja es que reorganizar muchas fichas puede requerir muchos movimientos: en el peor caso, $O(n^2)$.

- **Ordenamiento avanzado: Merge Sort.** Si se digitaliza el catálogo completo, se puede dividir en grupos, ordenar cada grupo y unirlos. Tiene un tiempo O(n log n), por lo que conviene para colecciones grandes. Su desventaja es que necesita memoria adicional; en una implementación común con arreglos, O(n).

- **Tabla hash.** En una versión digital se usaría el identificador único del ejemplar como clave y su ubicación como valor. Así se localiza directamente un ejemplar. Una consulta tarda O(1) en promedio bajo condiciones adecuadas, pero puede llegar a O(n) en el peor caso. Se necesita espacio adicional y resolver colisiones, que ocurren cuando distintas claves llegan a la misma posición. Tampoco permite recorrer los títulos en orden alfabético por sí sola.

Mejor opción. Para pocas fichas nuevas usaría inserción. Para un catálogo digital grande combinaría Merge Sort para generar listados y hash para búsquedas exactas por identificador. Cada uno resuelve una necesidad distinta.

Evidencia real. La Biblioteca del Congreso documenta el uso de catálogos por autor y tema [1]. Esto demuestra la necesidad histórica de organizar información. Inserción y Merge Sort son propuestas para este caso: la fuente no afirma que la biblioteca utilizara esos algoritmos. La aplicación real de

ordenamiento por mezcla y hash en software se observa en PostgreSQL, explicado en el segundo caso [3, 4].

2. Problema moderno: consultar productos de una tienda en línea

¿En qué consiste? En localizar un producto por código y mostrar listas ordenadas por precio cuando hay muchos registros.

¿Desde cuándo existe? Es un problema de la etapa del comercio electrónico, que se extendió con la web desde los años noventa. Aquí se analiza una tienda como ejemplo de aplicación, sin atribuirle una tecnología a una empresa específica.

¿Quiénes lo enfrentan? Clientes, empleados de inventario y desarrolladores de la tienda.

¿Qué dificultades presenta? Los precios y las existencias cambian, pueden existir productos con nombres parecidos y revisar todos los registros en cada consulta aumenta el trabajo del sistema.

¿Cómo se resolvía originalmente? En un negocio pequeño se podían consultar listas de precios y registros manuales. Una solución digital sencilla sería recorrer los productos uno por uno, pero esto se vuelve lento cuando crece el catálogo.

Relación con los algoritmos

- **Ordenamiento básico: burbuja.** Compara productos vecinos e intercambia sus posiciones si sus precios están al revés. Es sencillo de programar, pero su tiempo $O(n^2)$ en el peor caso lo hace poco conveniente para catálogos grandes.

- **Ordenamiento avanzado: Merge Sort.** Ordena los productos por precio en $O(n \log n)$. Si la información no cabe en memoria, se pueden ordenar bloques en archivos y luego mezclarlos. La desventaja es el espacio adicional y, en la versión externa, las lecturas y escrituras en disco.

- **Tabla hash.** La clave sería el SKU o código único del producto y el valor sería su registro. Es útil para consultas exactas, con $O(1)$ promedio y $O(n)$ en el peor caso en una tabla hash común. Requiere memoria y manejo de colisiones. No sirve por sí sola para ordenar por precio ni para buscar todos los productos dentro de un rango.

Mejor opción. Combinaría hash para encontrar un producto por su código y ordenamiento por mezcla para generar listas por precio. En una base de datos real también consideraría un índice B-tree para consultas por rangos, según el tipo de consulta [3].

Evidencia real. PostgreSQL permite crear índices hash para comparaciones de igualdad [3]. Su código de ordenamiento explica que, para grandes cantidades de datos, utiliza archivos temporales y mezcla balanceada de varias vías [4]. Esto es una aplicación real de la familia de algoritmos de mezcla, aunque no es exactamente el Merge Sort de dos mitades visto en un ejemplo escolar. Estas fuentes muestran herramientas que podrían resolver el caso de la tienda; no prueban que todas las tiendas las usen.

Conclusión

El problema de fondo es parecido en ambos casos: tener muchos datos y necesitar encontrar algo rápido. Un algoritmo básico puede funcionar con pocos elementos, pero al crecer la cantidad conviene uno más eficiente. También hay que distinguir entre ordenar y buscar: una tabla hash ayuda a encontrar una clave exacta, mientras que un ordenamiento permite recorrer los datos siguiendo un criterio.

Fuentes

Consultadas el 11 de noviembre de 2026.

1. Library of Congress. *Origins and Development of the Card Catalog*. https://guides.loc.gov/card-catalog/card-catalog-history
2. Library of Congress. *About the Main Card Catalog*. https://www.loc.gov/research-centers/main/main-card-catalog/
3. PostgreSQL. *Index Types*. https://www.postgresql.org/docs/current/indexes-types.html
4. PostgreSQL. *Código fuente: tuplesort.c*. https://github.com/postgres/postgres/blob/master/src/backend/utils/sort/tuplesort.c

Anexo: ejemplo sencillo en Python

El archivo ejemplo_algoritmos.py incluye inserción, burbuja, Merge Sort y búsqueda con un diccionario de Python, que utiliza una tabla hash. Los datos son inventados para mostrar cómo funciona. El ejemplo compara los títulos y precios directamente, usando funciones y ciclos, sin lambda. Inserción ordena los libros por título; burbuja y Merge Sort ordenan los productos por precio.

```
# Ejemplos con datos inventados. Ejecutar con Python 3.

# Insercion: coloca cada libro en su lugar segun el titulo.
def insercion(libros):
    lista = libros.copy()
    for i in range(1, len(lista)):
        actual = lista[i]
        j = i - 1
        while j >= 0 and lista[j]['titulo'] > actual['titulo']:
            lista[j + 1] = lista[j]
            j = j - 1
        lista[j + 1] = actual
    return lista

# Burbuja: compara precios vecinos y los intercambia.
def burbuja(productos):
    lista = productos.copy()
    for pasada in range(len(lista) - 1):
        for i in range(len(lista) - 1 - pasada):
            if lista[i]['precio'] > lista[i + 1]['precio']:
                temporal = lista[i]
                lista[i] = lista[i + 1]
                lista[i + 1] = temporal
    return lista

# Merge Sort: divide la lista y despues une las partes ordenadas.
def merge_sort(productos):
    if len(productos) <= 1:
        return productos.copy()

    mitad = len(productos) // 2
    izquierda = merge_sort(productos[:mitad])
    derecha = merge_sort(productos[mitad:])
    resultado = []
    i = 0
    j = 0

    while i < len(izquierda) and j < len(derecha):
        if izquierda[i]['precio'] <= derecha[j]['precio']:
            resultado.append(izquierda[i])
            i = i + 1
        else:
            resultado.append(derecha[j])
            j = j + 1

    # Agregar los productos que quedaron pendientes.
    while i < len(izquierda):
        resultado.append(izquierda[i])
        i = i + 1
    while j < len(derecha):
        resultado.append(derecha[j])
        j = j + 1
    return resultado

libros = [
    {'id': 'L03', 'titulo': 'Pedro Páramo', 'ubicacion': 'A3'},
    {'id': 'L01', 'titulo': 'Don Quijote', 'ubicacion': 'A1'},
    {'id': 'L02', 'titulo': 'El principito', 'ubicacion': 'A2'},
]
productos = [
    {'sku': 'P03', 'nombre': 'Mouse', 'precio': 300},
    {'sku': 'P01', 'nombre': 'Teclado', 'precio': 500},
    {'sku': 'P02', 'nombre': 'Cable', 'precio': 100},
]

print('Libros por titulo:')
libros_ordenados = insercion(libros)
```

```
for libro in libros_ordenados:
    print(libro['titulo'])

print('Productos con burbuja:')
productos_ordenados = burbuja(productos)
for producto in productos_ordenados:
    print(producto['nombre'], producto['precio'])

print('Productos con Merge Sort:')
productos_ordenados = merge_sort(productos)
for producto in productos_ordenados:
    print(producto['nombre'], producto['precio'])

# Los diccionarios de Python usan tablas hash internamente.
# Guardar cada registro usando su codigo como clave.
libros_por_id = {}
for libro in libros:
    codigo = libro['id']
    libros_por_id[codigo] = libro

productos_por_sku = {}
for producto in productos:
    codigo = producto['sku']
    productos_por_sku[codigo] = producto

# Buscar por codigo sin recorrer toda la lista.
codigo_buscado = 'L02'
if codigo_buscado in libros_por_id:
    print('Libro encontrado:', libros_por_id[codigo_buscado])
else:
    print('Libro no encontrado')

codigo_buscado = 'P03'
if codigo_buscado in productos_por_sku:
    print('Producto encontrado:', productos_por_sku[codigo_buscado])
else:
    print('Producto no encontrado')
```